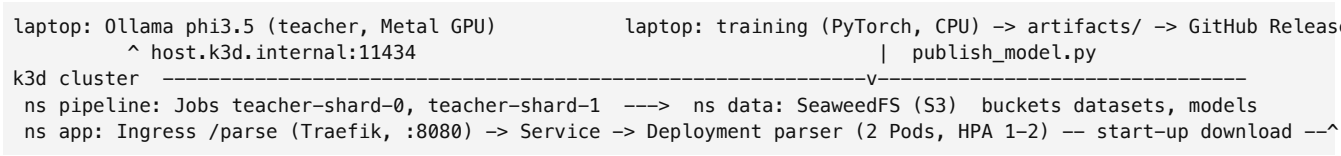
Knowledge Distillation and Deployment of a Command-Parsing Model on a Local Cloud

CS3081 Cloud Computing, Mini Project · Repository: `https://github.com/Nitheesh-Purnanand/cloud-computing-assignment`

Every number in this report is measured and comes from a file in `results/`; assumptions are marked **[ASSUMPTION]**. Hardware: one MacBook Air (Apple silicon, 10 cores, 16 GB), Docker via Colima (6 vCPU, 10 GB), k3d cluster `ccproj` (1 server + 1 agent).

Architecture



Task 1: Object storage and parallel teacher labelling

Setup. The cluster is created from `cluster/k3d.yaml` (port 8080 to Traefik, 8333 to the S3 NodePort). SeaweedFS runs as one Pod in namespace `data` (master, volume, filer and S3 gateway in one process, PVC on local-path). S3 credentials are generated into a git-ignored `.env` and loaded into Secrets by `scripts/secrets.sh`. `scripts/s3_setup.py` creates the buckets `datasets` and `models` and uploads `commands_pool.jsonl`. The teacher (phi3.5 in Ollama, temperature 0, seed 42) runs on the laptop to use its GPU; workers reach it through `host.k3d.internal`. The few-shot prompt is `prompts/teacher_prompt.txt` (examples taken only from the handout, none from validation).

Sharding. Worker *k* of *N* processes the commands whose 0-based line number *n* satisfies *n* mod *N* = *k*. Every *n* has exactly one remainder, so the shards are disjoint and together cover the file, with no coordination between workers. Round-robin assignment also balances the shards (1,500 each for *N* = 2) even if command difficulty drifts along the file. Each worker writes only its own object `datasets/teacher/shard-k.jsonl`, so there are no write conflicts. Progress is flushed every 25 records, and a restarted worker skips ids already stored, so a crash costs at most 25 teacher calls. Two Jobs (`teacher-shard-0/1`, `SHARD_COUNT=2`) ran at the same time in namespace `pipeline` (about 47 minutes, 0 restarts). Check: 3,000 ids in the shard outputs, 3,000 unique, identical to the pool.

Cleaning (`worker/clean.py`; rejects checked in the order a–e; output in id order; checksum `d0f4fc58…752d`, reproduced by `python worker/clean.py --local datasets/teacher`):

Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3,000	589	0	4	138	31	2,238

3,000 = 589 + 0 + 4 + 138 + 31 + 2,238. Nearly all (a) rejects are a correct JSON object followed by a note ("(Note: …)"); they are rejected because replies are stored exactly as generated.

Problem found. phi3.5 sometimes generated thousands of tokens (Ollama log: `n_gen = 2417`), which blocked both workers. I added `num_predict: 128` (a valid reply needs about 30 tokens), deleted the partial shards and relabelled everything, so all 3,000 replies come from one configuration.

Task 2: Training the student (sequence-level distillation)

Setup. `make train` runs `training/train.py`, which reads only `training/config.yaml` (seed 42, learning rate 3e-4, batch 16, 8 epochs, max length 96, library versions; the seed and deterministic algorithms are set in code). Each example is `Command: <text>\nJSON: <teacher reply><eos>`, and the loss covers the reply tokens only. The first 1,500 cleaned records are used; `make train SUBSET=50` trains on 50 (about 2.5 minutes). Loss per epoch is in `training/loss.csv` (0.619 → 0.009). v2 (`training/config_v2.yaml`: seed 43, 10 epochs) is in `training/loss_v2.csv`. Model and tokenizer files are in `s3://models/v1/` and `s3://models/v2/`. `scripts/publish_model.py` builds a deterministic tarball; its SHA-256 is stored as `models/v1/model.sha256` and equals the checksum of the GitHub Release asset `model-v1.tar.gz` (`4cffa3eb…85aa`; CI checks it with `sha256sum -c`).

Suitability of Pythia-70M. It is small enough to train on a CPU (about 1.5 minutes per epoch) and to serve within 1 CPU / 1 GiB, and its GPT-NeoX tokenizer handles JSON punctuation. It has no instruction tuning and little world knowledge, so untrained it produces no valid output at all (table below); everything it does here comes from fine-tuning on one narrow format.

Model (validation.jsonl, 100 commands)	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (`teacher_validation.json`)	75.0	67.0	55.0	65.0	72.0	41.0
Pythia-70M untrained (`pythia_base_validation.json`)	0.0	0.0	0.0	0.0	0.0	0.0
Student v1, deployed (`student_validation.json`)	100.0	88.0	76.0	85.0	85.0	48.0
Student v2 (`student_v2_validation.json`)	100.0	83.0	74.0	85.0	81.0	40.0

What changed. Fine-tuning took Pythia from 0% to 100% JSON validity. The student even beats the teacher on full frame (48 vs 41), mainly because it was trained only on replies that passed cleaning: it never learned the teacher's habit of adding notes (25% of teacher outputs on validation were invalid), and the copy-from-command filter removed invented slots. Where the teacher is consistently wrong (for example `date: "this afternoon"`), the student copies the mistake. v2 trained longer and fits the training set more closely (final loss 0.004) but scores lower, a sign of overfitting, so v1 is deployed.

Sequence-level versus logit-based distillation

- **Sequence-level** (Kim and Rush, 2016), used here: the student is trained with ordinary cross-entropy on the *text the teacher generated* (its decoded output sequence), treated as hard target tokens.
- **Logit-based** (word-level) distillation trains the student to match the teacher's *full probability distribution over the vocabulary* at every position (soft targets, usually a temperature-scaled KL divergence), which also carries information about the alternatives the teacher considered.
- **Why it is not possible here:** Ollama returns only the generated text, not per-token distributions. phi3.5 and Pythia also use different tokenizers and vocabularies (about 32k vs 50k tokens), so there is no token-by-token correspondence to match even if the logits were exported; and it would require running both models together during training.
- **Limitation of this approach:** the student can only be as good as the teacher's single greedy answer. Teacher errors that pass the cleaning filters (wrong intent, wrong but present slot text) are learned as truth, and the commands whose replies were invalid are simply lost, which biases the training data towards commands the teacher found easy.

Task 3: The inference service in a container

Design. FastAPI (`service/app.py`) on port 8000. At start-up a background thread downloads every file under `MODEL_URI` from `S3_ENDPOINT` (credentials from the Secret `s3-credentials` via `envFrom`) and loads the model. `/readyz` returns 503 until then, `/healthz` always 200; a failed load exits the process so Kubernetes restarts it. `POST /parse` runs greedy generation (`MAX_NEW_TOKENS`, one thread, one request at a time), keeps the first line and validates it against `schema.json` with `jsonschema`. Invalid output returns HTTP 500 `{"error": "invalid_model_output"}`, never broken JSON with 200. Headers: `X-Model-Version` = `MODEL_VERSION`, `X-Served-By` = Pod hostname. `FAULT_INJECT=1` replaces the output with `not json` before validation.

Image. `python:3.11.10-slim`, CPU-only PyTorch wheel, every package pinned (`service/requirements.txt` is the full `pip freeze`), no weights, non-root user; 1.27 GB. Built with one command: `make image`.

Measured (`results/limits_test.txt`, `results/fault_inject.txt`): `docker run --cpus 1 --memory 1g`, 50 sequential requests: 50 × HTTP 200, `OOMKilled: false`, **peak memory 576 MiB** (sampled with `docker stats`). With `FAULT_INJECT=1` in the cluster: HTTP 500 with the exact error body.

Task 4: Kubernetes deployment, scaling and reliability

Manifests (`k8s/30-app.yaml`): Deployment `parser` (2 replicas; requests 500m / 512Mi, limits 1 CPU / 1Gi; liveness `/healthz`; readiness `/readyz` every 2 s; `maxSurge 1, maxUnavailable 0`; a 5 s preStop sleep so endpoints drop a Pod before it stops), ClusterIP Service, Traefik Ingress for `/parse`, and an HPA (min 1, max 2, 70% CPU of the request). Each test is reproduced by a script in `scripts/`.

Test	Result (measured)	Evidence
Readiness gating	The EndpointSlice lists no ready endpoint for about 2 s after each Pod starts, until the model is loaded	`cold_start.txt`
Cold start (3 runs)	2 s, 2 s, 2 s from Pod creation to Ready; **median 2 s**. Logs: `downloaded model v1 (8 files) from s3://models/v1/ … in 0.4s`	`cold_start.txt`
HPA	From 1 replica under load: `SuccessfulRescale New size: 2`; after the load stopped, back to 1 replica (cpu 0%/70%)	`hpa_test.txt`
Rolling update v1→v2, then `rollout undo`	395 requests, 1 failed (**0.25%**, an HTTP 500 invalid-output answer, not a dropped connection); 94 served by v2; after the undo every response has `X-Model-Version: v1`	`rolling_update.txt`
Recovery	One Pod deleted under load: ready replicas 1 → 2 in about 7 s; 192 requests, 0 failed	`recovery.txt`
Load balancing (2 Ready, standard command)	112 and 112 successful requests (50% each)	`loadtest_headers.json`

Limitations. The whole cluster runs on one laptop inside one Docker VM. The two “nodes” are containers on the same machine, so this shows Pod-level self-healing, not fault tolerance across machines; losing the laptop loses everything, including the single-replica SeaweedFS. Cold start is short because the image is pre-imported on the nodes and the 280 MB model comes from in-cluster storage **[ASSUMPTION: a registry pull and remote storage would add tens of seconds]**. metrics-server needs about a minute before the HPA sees CPU (`FailedGetResourceMetric` events at start). With at most 2 replicas and a 1-CPU limit, scaling can only double capacity. To get exactly two Ready replicas for a test, the scripts set HPA `minReplicas` to 2 temporarily and restore it afterwards.

Task 5: Subnetting and network access control

Subnet	Hosts	Network / prefix	Usable range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

The working is in `network/subnets.md` (smallest prefix with $2^{32-p} - 2 \geq$ hosts, largest first, contiguous). NACL for the public subnet (`network/nacl.json`): inbound 100 deny all from 198.51.100.0/24; 110 allow TCP 80 and 120 allow TCP 443 from 0.0.0.0/0; 130 allow TCP 22 from 10.20.5.128/27; 140 allow TCP 1024–65535 from 10.20.0.0/22 (app replies). Outbound 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Everything else hits the default deny. `nacl_sim.py --check` on the sample packets: 6/6.

Why replies need their own rules. A NACL is stateless: it does not remember that a connection was opened, so every packet, including a reply, is checked on its own. The response to a client's request on port 443 leaves towards the client's ephemeral port, and the app's answer to the load balancer arrives on the load balancer's ephemeral port, so both directions need explicit rules for 1024–65535. **Why order matters.** Rules are evaluated from the lowest number and the first match wins. A packet from 198.51.100.9 to port 443 also matches allow rule 120; the deny works only because its number (100) is lower than every allow rule it must override.

Task 6: Reproducibility, testing and CI/CD

`make up` (venv, cluster, SeaweedFS with buckets and data, service image imported into k3d, models v1/v2 into S3 from local artifacts or the GitHub Releases, deploy) brings the system up after `git clone` and `cd`: three commands in total. `make down` runs `k3d cluster delete`. The workflow `.github/workflows/ci.yml` runs **lint** (ruff) → **unit tests** (8 pytest tests: rejection reasons and counts, duplicates, shard coverage, API headers, `FAULT_INJECT`, schema rejection, probes) → **evaluate** (downloads the model tarball from the GitHub Release, checks its SHA-256, runs `predict.py` and `score.py` on `validation.jsonl`) → **quality gate** (`score.py --min-validity 95`), plus a **gitleaks** scan of the full history (no leaks). The run for `model-v1` passed every stage. The gate was shown working with the untrained Pythia-70M, published as release `model-bad`: run `https://github.com/Nitheesh-Purnanand/cloud-computing-assignment/actions/runs/42552350446` passes lint, tests and evaluation and fails at the quality gate with `QUALITY GATE FAILED: json_validity 0.0 < 95.0`. AI use is described in `AI_USE.md`.

Task 7: Performance, cost and review

Measured with the standard Locust command through the Ingress, 2 Ready replicas (`results/loadtest_stats.csv`, `loadtest_top_pods.txt`): 222 requests (CSV; the console summary shows 224), 0 failures (**error rate 0%**), **p50 2,100 ms, p95 2,500 ms, 1.87 requests/s**; each Pod about 997m CPU (at its 1-core limit) and 464–545 MiB memory; cold start median 2 s (Task 4).

Cost (rate card, billed on limits): one replica = 1 vCPU × 3.50 + 1 GiB × 0.45 = 3.95 INR/h; two replicas = 7.90 INR/h. Requests per hour = 1.8713 × 3,600 = 6,737. Cost per 1,000 requests = 7.90 / 6,737 × 1,000 = **1.17 INR**. **[ASSUMPTION: only the parser replicas are billed; SeaweedFS, the control plane and the teacher are excluded.]**

SLO. 99% of `POST /parse` requests return HTTP 200 within 30 s, over one standard load-test run. Error budget = 1% × 222 = 2.22, so 2 failed requests are allowed; 0 were used (0% of the budget). In the rolling-update run the same SLO allows 3 failures out of 395; 1 was used (33%).

Concern	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	589 of 3,000 teacher replies (19.6%) rejected as invalid JSON	A fifth of the teacher time is wasted; the single-Pod SeaweedFS was OOMKilled at 768 Mi during a 280 MB upload	A stop sequence or JSON-constrained decoding for the teacher; SeaweedFS with separate, replicated volume servers
Model quality	Full frame 48% (teacher 41%); date accuracy 76%	Slot boundaries are learned from a noisy teacher; consistent teacher errors are copied	A better prompt or larger teacher, more cleaned data, a check that slot text occurs in the command
Latency and throughput	p95 2,500 ms, 1.87 req/s with 2 replicas	One request at a time per Pod; greedy generation on one CPU	Request batching, int8 or ONNX Runtime inference, stopping generation at the first newline
Resource utilisation	About 997m CPU and ≤ 545 MiB per Pod under load; 576 MiB peak at a 1 GiB limit	CPU-bound at the limit while about half the memory limit is unused	Lower the memory limit to 768 Mi; autoscale on request rate or latency instead of CPU
Reliability and recovery	0.25% failed during the rollout; replica restored in about 7 s with 0 failures	Single machine: no protection against node or disk loss	Multi-node cluster, PodDisruptionBudget, Pod anti-affinity, replicated storage
Security and cost	1.17 INR per 1,000 requests; gitleaks: no leaks	One static S3 admin key is shared by every namespace; idle replicas are still billed	Per-namespace least-privilege S3 identities; scale to zero when idle

References

Kim and Rush, Sequence-Level Knowledge Distillation, EMNLP 2016. Bastianelli et al., SLURP, EMNLP 2020 (data, CC BY 4.0). EleutherAI, Pythia-70M model card. Documentation of k3d, k3s, Kubernetes, SeaweedFS, Hugging Face Transformers, FastAPI, Locust, GitHub Actions and gitleaks (links in the project handout).